HalluRAG: A Lightweight Framework for Hallucination Type Classification and Self-Correction in Retrieval-Augmented Generation Systems

Mini Project report submitted
in
Partial fulfilment of requirement for the award of
Degree of

Master of Technology
in
Computer Science and Engineering

by

Ms. Mrunali Tirpude

(MTech CSE Sem II)

Guide

Dr. Mangala Madankar

(MTech CSE Sem II)

Department of Computer Science and Engineering

G H Raisoni College of Engineering, Nagpur
(An Empowered Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University, Nagpur)

May 2026

DECLARATION

We, hereby declare that the project report titled "HalluRAG: A Lightweight Framework for Hallucination Type Classification and Self-Correction in Retrieval-Augmented Generation Systems" submitted herein has been carried out by us in the partial fulfillment of requirement for the award of degree Master of Technology in Computer Science and Engineering. The work is original and has not been submitted earlier as a whole or in part for the award of any degree / diploma at this or any other Institution / University.

We also hereby assign to G H Raisoni college of Engineering, Nagpur all rights under copyright that may exist in and to the above work and any revised or expanded derivatives works based on the work as mentioned. Other work copied from references, manuals etc. are disclaimed.

Place: Nagpur **Mrunali Tirpude**
Date: **(Name and Signature)**

Certificate

The project report titled "**HalluRAG: A Lightweight Framework for Hallucination Type Classification and Self-Correction in Retrieval-Augmented Generation Systems**" submitted by **Mrunali Tirpude** for the award of degree of Master of Technology in Computer Science and Engineering has been carried out under our supervision. The work is comprehensive, complete and fit for evaluation.

Dr. Mangala Madankar
HOD
Department of Computer Science and Engineering
GHRCE, Nagpur

Dr. Prasad P. Lokulwar
PG Coordinator
Department of Computer Science and Engineering
GHRCE, Nagpur

Dr. Mangala Madankar
HOD
Department of Computer Science and Engineering
GHRCE, Nagpur

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to all those who have supported and guided me throughout this mini project work.

First and foremost, I am deeply grateful to my project guide, Dr. Mangala Madankar, HOD, Department of Computer Science and Engineering, GHRCE Nagpur, for her invaluable guidance, constant encouragement, and constructive feedback throughout the development of this project. Her expertise and insights have been instrumental in shaping this work.

I am also grateful to Dr. Prasad P. Lokulwar, PG Coordinator, Department of Computer Science and Engineering, for his administrative support and valuable suggestions during the course of this project.

I extend my sincere thanks to the entire faculty of the Department of Computer Science and Engineering, GHRCE Nagpur, for their encouragement and technical inputs during various stages of this project.

I would also like to acknowledge the contributions of the open-source community behind LangChain, HuggingFace, FAISS, and the Groq API, whose freely available tools made this research accessible and implementable on standard hardware.

Finally, I express my heartfelt gratitude to my family for their unwavering support, encouragement, and patience throughout this academic journey.

Ms. Mrunali Tirpude

ABSTRACT

Retrieval-Augmented Generation (RAG) has emerged as a widely adopted technique for building intelligent document question-answering systems, wherein a language model retrieves contextually relevant chunks from an uploaded document and generates grounded answers. Despite its advantages over purely parametric models, RAG systems continue to suffer from hallucination — the generation of confident but factually incorrect, unsupported, or misleading responses. Existing research has addressed hallucination detection in isolation, but no unified framework currently integrates detection, type classification, and automated self-correction within a single RAG pipeline, particularly in lightweight, resource-constrained environments.

This project proposes HalluRAG — a lightweight, domain-agnostic framework that addresses this gap through three novel contributions: (1) sentence-level hallucination detection using a DeBERTa-based Natural Language Inference (NLI) model integrated directly into the RAG pipeline; (2) a novel four-class hallucination taxonomy specifically designed for document-grounded RAG systems, classifying errors as Intrinsic (answer contradicts the document), Extrinsic (answer adds facts absent from the document), Numerical (wrong figures or dates), or Omission (critical facts omitted from the answer); and (3) an iterative type-specific self-correction loop that automatically constructs targeted re-prompting instructions based on the detected hallucination type and re-queries the language model until the faithfulness score meets an acceptable threshold.

The system is implemented entirely using free, open-source tools including LangChain for pipeline orchestration, FAISS for semantic vector retrieval, Llama 3 via the Groq API for answer generation, and the DeBERTa-v3-base cross-encoder model from HuggingFace for NLI-based faithfulness scoring. The complete pipeline is designed to run on a standard laptop with 8GB RAM and no GPU requirement, making it practically accessible for real-world deployment in legal, medical, academic, and corporate document Q&A applications.

The project is structured across three development phases: Phase 1 (Semester II Mini Project) delivers the core RAG pipeline with a Streamlit web interface; Phase 2 (Semester III) integrates the hallucination detection and type classification modules; and Phase 3 (Semester IV) completes the self-correction loop and targets publication in a Scopus-indexed journal. Evaluation will be performed using the RAGAS framework, HaluEval benchmark, and TruthfulQA dataset.

Keywords: Retrieval-Augmented Generation, Hallucination Detection, Natural Language Inference, DeBERTa, Self-Correction, LangChain, FAISS, Llama 3, Faithfulness Scoring, Domain-Specific Q&A.

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Fig. 3.1	Hallucination Type Classification Decision Tree	12
Fig. 4.1	HalluRAG System Architecture — 7-Step Pipeline	16
Fig. 5.1	DeBERTa NLI Faithfulness Scoring Process	18
Fig. 5.2	Self-Correction Loop Flow Diagram	19
Fig. 5.3	Streamlit Wireframe	20
Fig. 6.1	HalluRAG Module Interaction Diagram	21

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Comparative Analysis of Literature Review Papers	8-9
Table 3.1	Research Gap — Existing Papers vs HalluRAG	10
Table 3.2	HalluRAG Hallucination Type Classification Rules	11
Table 4.1	Software and Hardware Specifications	14
Table 6.1	Module Description Summary	21
Table 7.1	Phase-wise Implementation Plan	26

LIST OF ABBREVIATIONS

Abbreviation	Full Form
RAG	Retrieval-Augmented Generation
NLI	Natural Language Inference
LLM	Large Language Model
FAISS	Facebook AI Similarity Search
DeBERTa	Decoding-enhanced BERT with Disentangled Attention
BERT	Bidirectional Encoder Representations from Transformers
API	Application Programming Interface
GPU	Graphics Processing Unit
CPU	Central Processing Unit
UI	User Interface
Q&A	Question and Answer
NLP	Natural Language Processing
RAGAS	Retrieval Augmented Generation Assessment
F1	F1 Score (Harmonic Mean of Precision and Recall)
HOD	Head of Department
GHRCE	G H Raisoni College of Engineering
RTMNU	Rashtrasant Tukadoji Maharaj Nagpur University
ESWA	Expert Systems with Applications
MDPI	Multidisciplinary Digital Publishing Institute

INDEX

SR.NO.	TITLE	Page No.
1	Declaration	i
2	Certificate	ii
3	Acknowledgement	iii
4	Abstract	iv
5	List of Figures	v
6	List of Tables	v
7	List of Abbreviations	vi
8	**CHAPTER 1: INTRODUCTION**	**1-3**
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Project Objectives	2
	1.4 Scope of the Project	2
	1.5 Organization of the Report	3
9	**CHAPTER 2: LITERATURE REVIEW**	**4-9**
	2.1 Introduction to Literature Review	4
	2.2 RAG Core Papers	4
	2.3 Hallucination Detection Papers	5
	2.4 NLI Models for Faithfulness	6
	2.5 Domain-Specific RAG Papers	7
	2.6 Recent Papers (2024–2025	8
	2.7 Comparative Analysis of Literature	8
	2.8 Summary	9
10	**CHAPTER 3: METHODOLOGY**	**10-13**
	3.1 Research Gap Identification	10
	3.2 Proposed Methodology Overview	10
	3.3 Evaluation Methodology	13
11	**CHAPTER 4: DATA COLLECTION / TOOLS / PLATFORMS USED**	**14-16**
	4.1 Development Environment	14
	4.2 Python Libraries and Frameworks	14
	4.3 Datasets and Benchmarks	15

SR.NO.	TITLE	Page No.
	4.4 Folder Structure	16
12	**CHAPTER 5: DESIGN / IMPLEMENTATION**	**17-20**
	5.1 System Architecture	17
	5.2 DeBERTa NLI Integration	17
	5.3 Self-Correction Prompt Templates	18
	5.4 Streamlit Web Interface	19
13	**CHAPTER 6: MODULE DESCRIPTION**	**21-24**
	6.1 Overview of System Modules	21
	6.2 Module 01: Document Ingestion Module	22
	6.3 Module 02: RAG Query and Generation Module	22
	6.4 Module 03: Hallucination Detection Module	22
	6.5 Module 04: Type Classification Module	23
	6.6 Module 05: Self-Correction Module	23
	6.7 Module 06: Streamlit Web Interface Module	24
14	**CHAPTER 7: TESTING AND SUMMARY OF RESULTS**	**25-27**
	7.1 Testing Strategy	25
	7.2 Expected Results	26
	7.3 Comparative Analysis	27
15	**CHAPTER 8: APPLICATIONS**	**28-29**
	8.1 Overview	28
	8.2 Legal Document Question Answering	28
	8.3 Medical Report and Drug Guide Analysis	28
	8.4 Academic Research Assistance	28
	8.5 Corporate Knowledge Base Q&A	29
	8.6 Government Document and RTI Q&A	29
	8.7 Educational Content and E-Learning	29
16	**CONCLUSION AND FUTURE SCOPE**	**30-31**
	Conclusion	30
	Summary of Phase 1 Deliverables	31
	Future Scope	31
17	**REFERENCES**	**32-33**

CHAPTER 1: INTRODUCTION

1.1 Background

The rapid advancement of Large Language Models (LLMs) has revolutionized natural language processing, enabling AI systems to engage in complex reasoning, answer questions, and generate human-like text across diverse domains. However, a fundamental limitation of purely parametric models — those that rely solely on knowledge encoded in their weights during training — is their inability to access up-to-date, document-specific, or domain-specific information. This limitation manifests as knowledge staleness, factual inconsistency, and hallucination, wherein the model generates confident but incorrect or fabricated information.

To address this limitation, Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. in 2020 as a paradigm that combines information retrieval with language generation. In a RAG system, when a user poses a question, the system first retrieves the most relevant passages from an external knowledge source — such as an uploaded PDF document — and then conditions the language model's generation on those retrieved passages. This grounding mechanism substantially improves factual accuracy by anchoring the model's responses to specific source documents rather than relying on memorized training knowledge.

RAG systems have found widespread application in domain-specific Q&A scenarios, including legal document analysis, medical report interpretation, academic research assistance, and corporate knowledge management. Frameworks such as LangChain and LlamaIndex have further democratized RAG development by providing modular, easy-to-use pipeline components that enable rapid prototyping and deployment.

1.2 Problem Statement

Despite the significant advantages of RAG over purely parametric generation, hallucination — the generation of plausible but factually incorrect or unsupported content — remains a persistent challenge. Studies have reported hallucination rates of 15-20% even in well-designed RAG systems (Siriwardhana et al., 2023). This problem is particularly critical in high-stakes domains such as medicine, law, and finance, where a single hallucinated fact can lead to dangerous or costly consequences.

A comprehensive review of existing research reveals three critical gaps in the current state of the art:

- No existing RAG framework integrates hallucination detection as an inline component of the generation pipeline. Detection methods such as TRUE (Honovich et al., 2022) and

SelfCheckGPT (Manakul et al., 2023) operate as post-hoc evaluation tools, not as real-time pipeline components.

- No existing work classifies hallucinations into specific types within a RAG context. Binary detection (hallucinated vs. faithful) provides insufficient diagnostic information for targeted correction. Understanding whether an error is an intrinsic contradiction, an extrinsic fabrication, a numerical error, or an omission is essential for applying the appropriate corrective action.
- Automated self-correction of detected hallucinations remains an open research problem. A 2025 MDPI survey on hallucination mitigation in retrieval-augmented LLMs explicitly identifies post-hoc self-correction as the least explored and most critical area requiring future research.

1.3 Project Objectives

The primary objectives of this project are as follows:

1. To design and implement a complete, end-to-end RAG pipeline for domain-specific document question answering using free, open-source tools.
2. To integrate a sentence-level hallucination detection module using the DeBERTa NLI model within the RAG pipeline.
3. To develop a novel four-class hallucination type classification system: Intrinsic, Extrinsic, Numerical, and Omission hallucinations.
4. To implement an iterative type-specific self-correction loop that automatically re-prompts the language model with targeted correction instructions based on the detected hallucination type.
5. To evaluate the complete system using standard benchmarks (RAGAS, HaluEval, TruthfulQA) and compare performance against baseline RAG without hallucination safeguards.
6. To deploy the system as a lightweight, accessible web application using Streamlit, operable on standard laptop hardware without GPU requirements.

1.4 Scope of the Project

The scope of this mini project (Phase 1) encompasses the design and implementation of the core RAG pipeline with document ingestion, vector-based retrieval, LLM-based answer generation, and a basic Streamlit web interface. The hallucination detection module (Phase 2) and the self-correction loop (Phase 3) will be developed in subsequent semesters as extensions of the current work.

The system is designed to be domain-agnostic — accepting any text-based document (PDF, DOCX, TXT) as input and answering questions based solely on the content of that document. This makes the framework applicable across legal, medical, academic, and corporate domains without domain-specific customization or retraining.

1.5 Organization of the Report

The remainder of this report is organized as follows: Chapter 2 presents a detailed literature review of relevant prior work. Chapter 3 defines the research problem and identifies the research gap. Chapter 4 describes the proposed system architecture and methodology. Chapter 5 details the tools, platforms, and hardware/software specifications. Chapter 6 provides a complete description of all system modules.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction to Literature Review

This chapter presents a comprehensive review of research papers relevant to the core components of the HalluRAG framework: Retrieval-Augmented Generation, hallucination detection in LLMs, Natural Language Inference models for faithfulness scoring, and domain-specific RAG applications. The review encompasses 15 seminal papers published between 2019 and 2025 and is organized into four thematic categories.

2.2 RAG Core Papers

2.2.1 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)

Lewis et al. (2020) introduced the foundational RAG framework, combining a pre-trained sequence-to-sequence generator (BART) with a dense passage retriever (DPR) through a joint fine-tuning approach. The paper introduced two RAG variants: RAG-Sequence (same retrieved passages across the full generated sequence) and RAG-Token (different passages per generated token). Evaluated on open-domain QA benchmarks including Natural Questions, WebQ, and TriviaQA, RAG significantly outperformed parametric-only models. However, the work does not address verification of answer faithfulness — the generator can still produce hallucinated content even when relevant documents are retrieved.

2.2.2 REALM: Retrieval-Augmented Language Model Pre-Training (Guu et al., 2020)

Guu et al. (2020) proposed training the retriever and language model jointly during pre-training, rather than only at inference time. REALM uses masked language modeling where predictions are conditioned on retrieved Wikipedia passages, requiring an asynchronous index refresh during training. REALM achieved state-of-the-art results on open-domain QA at the time of publication. Its primary limitation is computational cost — the training procedure requires massive GPU/TPU infrastructure, making it inaccessible for resource-constrained researchers.

2.2.3 Dense Passage Retrieval (Karpukhin et al., 2020)

Karpukhin et al. (2020) replaced traditional keyword-based retrieval (BM25) with dense neural retrieval using a bi-encoder architecture. Questions and passages are encoded into a shared dense vector space using separate BERT models, and relevant passages are retrieved using maximum inner product search via FAISS. DPR demonstrated 9-19% improvement over BM25 on top-20 passage retrieval accuracy across five QA datasets. DPR forms the retrieval backbone of modern RAG systems and is a core component in the proposed HalluRAG architecture.

2.2.4 RAGAS: Automated Evaluation of RAG (Es et al., 2023)

Es et al. (2023) introduced RAGAS, the first comprehensive automated evaluation framework specifically designed for RAG pipelines. RAGAS provides four reference-free metrics: Faithfulness (is the answer grounded in the retrieved context?), Answer Relevancy (does the answer address the question?), Context Precision (are retrieved chunks relevant?), and Context Recall (does the context contain sufficient information?). RAGAS demonstrated that high faithfulness and high answer relevancy are independent properties — a system can be faithful but irrelevant. The framework has become the industry standard for RAG evaluation and is used as the primary evaluation metric for Phase 1 of the proposed project.

2.3 Hallucination Detection Papers

2.3.1 TruthfulQA (Lin et al., 2022)

Lin et al. (2022) introduced TruthfulQA, a benchmark of 817 questions across 38 categories designed to test whether LLMs repeat common human misconceptions. Results showed that GPT-3 (175B parameters) achieved only 58% truthfulness, worse than a trivial baseline. Critically, larger models showed higher tendency to reproduce false beliefs confidently. While TruthfulQA established the importance of truthfulness evaluation, it tests general world-knowledge truthfulness rather than document-grounded RAG faithfulness, limiting its direct applicability to the proposed work.

2.3.2 HaluEval (Li et al., 2023)

Li et al. (2023) published HaluEval, a large-scale hallucination evaluation benchmark comprising 35,000 LLM-generated samples across question answering, dialogue, and summarization tasks. ChatGPT-generated hallucinated versions of real examples were created and verified by human annotators. Experimental results revealed that ChatGPT correctly identified only 19.5% of hallucinated QA answers when provided with a knowledge passage. HaluEval demonstrated that hallucination is task-dependent and that providing source documents significantly aids detection. However, HaluEval provides only binary labels (hallucinated/not hallucinated) without type classification or correction mechanisms.

2.3.3 TRUE: Re-evaluating Factual Consistency Evaluation (Honovich et al., 2022)

Honovich et al. (2022) proposed using Natural Language Inference (NLI) as a principled approach to faithfulness evaluation. The TRUE framework treats source text as a premise and generated sentences as hypotheses, using a trained NLI model to determine entailment. Evaluated across 11 datasets spanning summarization, dialogue, and QA, TRUE (using DeBERTa-large-mnli) outperformed all other automatic faithfulness metrics including ROUGE, BERTScore, and QA-based approaches. This work directly informs the hallucination detection methodology of the proposed HalluRAG system.

2.3.4 SelfCheckGPT (Manakul et al., 2023)

Manakul et al. (2023) introduced SelfCheckGPT, a zero-resource hallucination detection method based on self-consistency: information that the model generates consistently across multiple samples is likely factual, while inconsistent information is likely hallucinated. Three consistency checking variants were proposed: BERTScore, NLI-based, and n-gram overlap. While SelfCheckGPT requires no external knowledge source, it demands 5-20 model samples per query (5-20x inference overhead) and performs poorly in RAG settings where document grounding artificially increases generation consistency regardless of truthfulness.

2.3.5 FActScore (Min et al., 2023)

Min et al. (2023) proposed FActScore, decomposing generated text into atomic facts and verifying each against a knowledge source via retrieval and NLI. FActScore demonstrated that ChatGPT achieves only 58% atomic fact accuracy on biography generation. While highly precise, FActScore requires 20-30 separate API calls per answer (one per atomic fact), making it computationally prohibitive for real-time RAG applications.

2.4 NLI Models for Faithfulness

2.4.1 DeBERTa (He et al., 2021)

He et al. (2021) introduced DeBERTa, improving upon BERT and RoBERTa through two key innovations. First, disentangled attention separates word content and positional information into independent vectors, computing three attention types (content-to-content, content-to-position, position-to-content) independently. Second, an Enhanced Mask Decoder injects absolute position information during masked token prediction. Additionally, Scale-invariant Fine-Tuning (SiFT) with virtual adversarial training improves generalization. DeBERTa surpassed human performance on SuperGLUE for the first time and achieves the highest NLI accuracy among free open-source models, making it the optimal choice for the faithfulness scoring module in HalluRAG.

2.4.2 BERT (Devlin et al., 2019)

Devlin et al. (2019) introduced BERT, which revolutionized NLP through bidirectional pre-training on masked language modeling and next sentence prediction tasks. BERT's Transformer encoder architecture enables deep contextualized word representations that capture both semantic and syntactic information simultaneously. BERT established the pre-train/fine-tune paradigm that underlies all subsequent NLI models including DeBERTa.

2.4.3 FactCC (Kryscinski et al., 2020)

Kryscinski et al. (2020) proposed FactCC, a BERT-based NLI classifier trained to detect factual inconsistencies between source documents and abstractive summaries. Using six automated transformation types (entity swap, pronoun swap, number swap, sentence negation, noise insertion, paraphrase insertion) for data augmentation, FactCC achieved 86% accuracy at identifying factually inconsistent summaries. The six transformation types in FactCC directly inspired the four hallucination types (Intrinsic, Extrinsic, Numerical, Omission) defined in the proposed HalluRAG taxonomy.

2.5 Domain-Specific RAG Papers

2.5.1 RAG vs. Fine-tuning (Siriwardhana et al., 2023)

Siriwardhana et al. (2023) systematically compared RAG against fine-tuning on a domain-specific agricultural Q&A dataset. Results demonstrated that RAG outperforms fine-tuning for factual Q&A as it can always reference source documents, while fine-tuning performs better for domain vocabulary acquisition. The hybrid approach (RAG + fine-tuning) achieved best overall performance. Critically, even the best-performing RAG configuration hallucinated on 15-20% of test questions — directly motivating the need for integrated hallucination detection.

2.5.2 MedRAG (Xiong et al., 2024)

Xiong et al. (2024) published MedRAG, a systematic benchmark evaluating RAG configurations for medical QA across four benchmarks (MedQA, MedMCQA, PubMedQA, MMLU-Medical), seven corpora, and six LLMs. The best configuration outperformed standard prompting by 18% on medical QA accuracy. Despite this improvement, all RAG configurations still produced significant hallucination errors on complex medical reasoning questions, underscoring the need for hallucination detection even in optimized RAG systems.

2.5.3 LlamaIndex (Liu, 2022)

Liu (2022) introduced LlamaIndex, an open-source framework providing ready-to-use components for RAG pipeline development including document loaders, text splitters, vector indices, and query engines. LlamaIndex significantly reduces RAG development time from weeks to hours and supports integration with 150+ data sources and 30+ LLMs. Notably, LlamaIndex provides no built-in hallucination detection or verification — this gap directly motivates the development of HalluRAG as a hallucination safety layer.

2.6 Recent Papers (2024-2025)

2.6.1 ReDeEP (Sun et al., ICLR 2025)

Sun et al. (2025) investigated the internal mechanisms underlying RAG hallucinations using mechanistic interpretability. The work discovered that hallucinations specifically occur when Knowledge Feed-Forward Networks (FFNs) in later LLM layers over-add parametric memory to the residual stream, while Copying Heads fail to effectively integrate external context. ReDeEP proposes decoupling these components for more accurate detection. While theoretically rigorous, ReDeEP requires white-box access to model internals, making it inapplicable to API-based LLMs.

2.6.2 MDPI Survey on Hallucination Mitigation (2025)

This comprehensive survey examined hallucination causes, detection methods, and mitigation strategies across the entire RAG pipeline. The survey confirmed that detection methods (TRUE, SelfCheckGPT, FActScore) are more mature than mitigation approaches, and explicitly identified post-hoc self-correction as the least explored and most needed research direction. This finding directly validates the research contribution of the proposed HalluRAG project.

2.6.3 FACTUM (Dassen et al., 2026)

Dassen et al. (2026) introduced FACTUM, a mechanistic framework for detecting citation hallucinations in long-form RAG using four internal model scores: Contextual Alignment Score (CAS), Attention Sink Usage Score (BAS), Parametric Force Score (PFS), and Pathway Alignment Score (PAS). FACTUM represents the most recent advancement in RAG hallucination detection but focuses specifically on citation verification rather than general content faithfulness classification and correction.

2.7 Comparative Analysis of Literature

Table 2.1: Comparative Analysis of Literature Review Papers

Paper	Year	RAG Pipeline	Hallucination Detection	Type Classification	Self-Correction	Lightweight
Lewis et al. (RAG)	2020	Yes	No	No	No	No
Karpukhin et al. (DPR)	2020	Partial	No	No	No	Yes
Es et al. (RAGAS)	2023	Partial	Partial	No	No	Yes
Lin et al. (TruthfulQA)	2022	No	Partial	No	No	Yes
Li et al. (HaluEval)	2023	No	Yes	No	No	Partial

Honovich et al. (TRUE)	2022	No	Yes	No	No	Partial
Manakul et al. (SelfCheck)	2023	No	Yes	No	No	No
Min et al. (FActScore)	2023	No	Yes	No	No	No
He et al. (DeBERTa)	2021	No	Partial	No	No	Yes
Devlin et al. (BERT)	2019	No	No	No	No	Yes
Siriwardhana et al.	2023	Yes	No	No	No	Partial
Xiong et al. (MedRAG)	2024	Yes	No	No	No	No
Sun et al. (ReDeEP)	2025	No	Yes	No	No	No
MDPI Survey	2025	No	Partial	No	No	Yes
HalluRAG (Proposed)	**2026**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

2.8 Summary

The literature review reveals a clear and consistent gap across all reviewed works: while RAG pipelines are well-established and hallucination detection methods exist, no single work integrates detection within a RAG pipeline, classifies hallucination types, or provides automated type-specific correction. Furthermore, most existing detection approaches are computationally expensive and impractical for real-time deployment. The proposed HalluRAG framework directly addresses all these limitations.

CHAPTER 3: METHODOLOGY

3.1 Research Gap Identification

Based on the comprehensive literature review presented in Chapter 2, the following research gap has been formally identified:

"Existing Retrieval-Augmented Generation systems detect hallucinations but cannot classify their specific type or automatically correct them — particularly in a lightweight, domain-agnostic framework accessible without expensive GPU infrastructure or paid APIs. A 2025 MDPI survey confirms that automated self-correction remains an open, unsolved research problem."

This gap is characterized by four distinct dimensions, each supported by evidence from the literature:

Table 3.1: Research Gap Analysis — Existing Papers vs. HalluRAG

Dimension	Existing Research	HalluRAG Contribution
Integrated Pipeline	Detection tools operate as external evaluators, not pipeline components	DeBERTa NLI integrated inline within RAG pipeline
Type Classification	Binary: hallucinated / not hallucinated only (HaluEval, TRUE)	4-class: Intrinsic / Extrinsic / Numerical / Omission
Self-Correction	Identified as open problem (MDPI 2025) — no solution exists	Type-specific iterative re-prompting loop (max 3 attempts)
Lightweight Deployment	FActScore: 30 API calls/answer; SelfCheck: 20x inference	Single-pass DeBERTa check; runs on 8GB RAM laptop

3.2 Proposed Methodology Overview

The HalluRAG framework employs a four-stage methodology that addresses each dimension of the identified research gap:

3.2.1 Methodology 1: Retrieval-Augmented Generation Pipeline

The RAG pipeline is implemented using LangChain as the orchestration framework. Document ingestion involves loading text-based documents using LangChain's PyPDFLoader, splitting content into chunks of 300 words with a 50-word overlap using RecursiveCharacterTextSplitter, and converting chunks to 768-dimensional dense vectors using the sentence-transformers model (all-MiniLM-L6-v2). Vectors are stored in a FAISS CPU index for efficient similarity search.

At query time, the user's question is embedded into the same vector space, and the top-5 most similar chunks are retrieved using cosine similarity. These chunks, together with the user's question, are formatted

into a context-grounded prompt and sent to Llama 3 (llama3-8b-8192) via the free Groq API. The LLM is explicitly instructed to generate answers exclusively from the provided context, reducing but not eliminating hallucination.

3.2.2 Methodology 2: Sentence-Level Hallucination Detection Using DeBERTa NLI

The generated answer is split into individual sentences. For each sentence, the DeBERTa NLI model (cross-encoder/nli-deberta-v3-base) is applied with the retrieved document chunk as the premise and the answer sentence as the hypothesis. DeBERTa computes three probability scores:

- ENTAILMENT score (E): probability that the sentence follows from the document
- NEUTRAL score (N): probability that the sentence is unrelated to or not contradicted by the document
- CONTRADICTION score (C): probability that the sentence contradicts the document

The faithfulness score for each sentence is defined as the ENTAILMENT probability. A sentence is flagged as hallucinated if its faithfulness score falls below the threshold $\tau = 0.75$. The overall answer faithfulness score is the mean faithfulness score across all sentences.

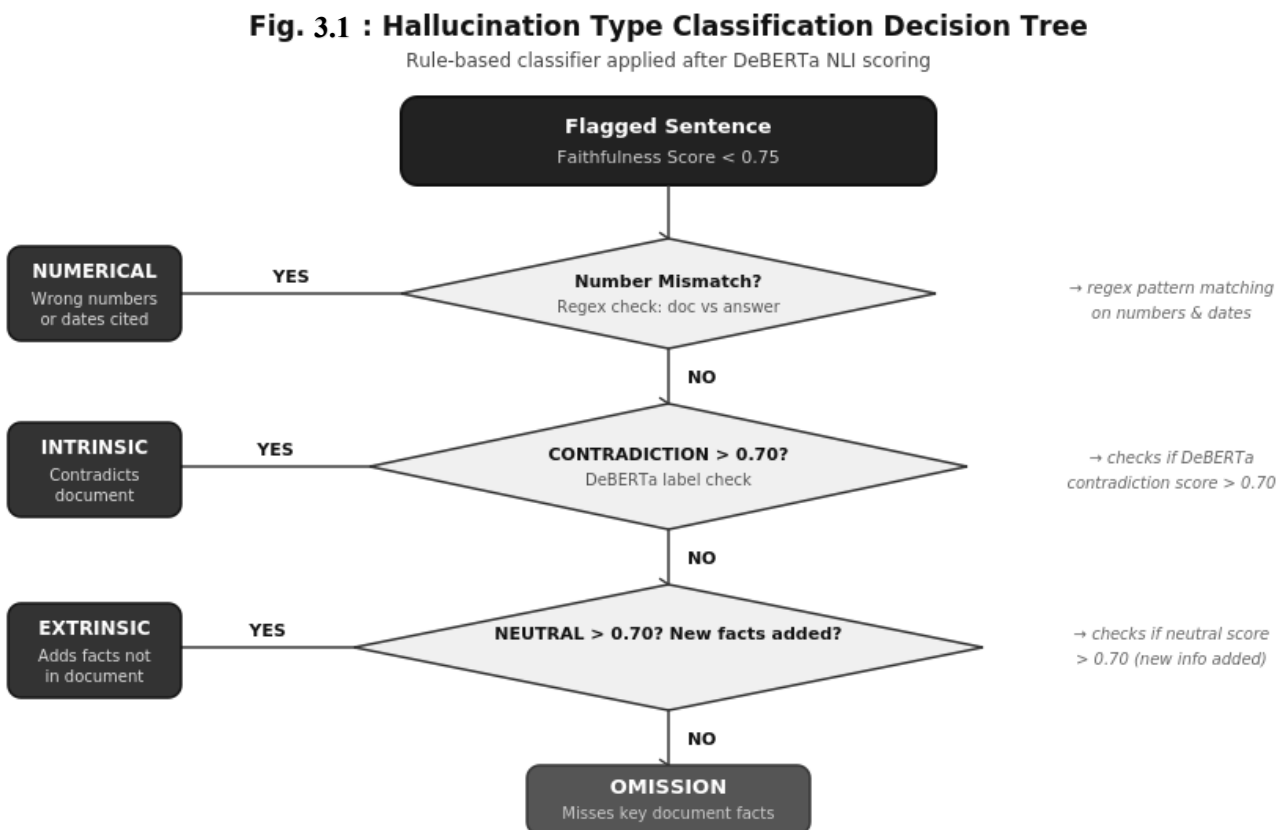
3.2.3 Methodology 3: Hallucination Type Classification

Upon detection of a hallucinated sentence (faithfulness score < 0.75), the system classifies the hallucination into one of four types:

Table 3.2: HalluRAG Hallucination Type Classification Rules

Type	Detection Condition	Meaning	Example
INTRINSIC	C > 0.70 AND no number mismatch	Answer directly contradicts document	Doc: 60% threshold; Answer: 50% threshold
EXTRINSIC	N > 0.70 AND new facts present	Answer adds info absent from document	Doc: no mention of day; Answer: 'first day of semester'
NUMERICAL	C or N > 0.70 AND number mismatch detected	Answer contains wrong numbers/dates	Doc: 7 days; Answer: 15 days
OMISSION	E > 0.75 but key document facts missing	Answer misses critical document facts	Doc: two rules; Answer mentions only one

Numerical hallucination is detected by extracting all numeric tokens (integers, decimals, percentages) from both the answer sentence and the retrieved chunk using regular expressions and checking for mismatches. Omission detection compares key noun phrases and numerical entities extracted from the retrieved chunks against those present in the generated answer.



3.2.4 Methodology 4: Iterative Type-Specific Self-Correction Loop

Upon classification of a hallucinated sentence, the self-correction module automatically constructs a type-specific correction prompt containing:

- The hallucinated sentence
- The specific error identified (wrong numbers, contradicted fact, fabricated information)
- The relevant document chunk containing the correct information
- An explicit instruction to rewrite only the flagged sentence using document-grounded information

This correction prompt is sent to Llama 3 via the Groq API. The rewritten sentence is re-evaluated by the DeBERTa NLI module. If the faithfulness score exceeds 0.75, the correction is accepted. If not, the loop repeats with a refined prompt, up to a maximum of three attempts. Sentences that cannot be corrected after three attempts are flagged with a warning badge for human review.

3.3 Evaluation Methodology

The HalluRAG system will be evaluated against three baseline configurations:

7. Baseline RAG: No hallucination detection or correction (standard pipeline)
8. RAG + Detection: Pipeline with DeBERTa faithfulness scoring, no correction
9. HalluRAG (Full): Complete pipeline with detection, type classification, and self-correction

Evaluation metrics include: Faithfulness Score (RAGAS), Answer Relevancy (RAGAS), F1 Score for hallucination detection (HaluEval benchmark), Accuracy on TruthfulQA subset, Hallucination rate reduction (percentage), and Average query processing time.

CHAPTER 4: DATA COLLECTION / TOOLS / PLATFORMS USED

4.1 Development Environment

Table 4.1: Software and Hardware Specifications

Specification	Details
Operating System	Windows 11 (64-bit)
Programming Language	Python 3.11
Code Editor	Visual Studio Code (VS Code)
RAM	8 GB DDR4
GPU	Not required — all models run on CPU
Python Version	3.11.x (recommended for library compatibility)
Virtual Environment	Python venv (isolated dependency management)
Browser	Any modern browser (Chrome/Firefox) for Streamlit UI

4.2 Python Libraries and Frameworks

The following open-source Python libraries are used in the project:

4.2.1 LangChain

LangChain is an open-source framework for building applications powered by large language models. It provides modular components for document loading, text splitting, embedding, vector storage, retrieval, and LLM chaining. In HalluRAG, LangChain serves as the orchestration layer connecting all pipeline components. Key components used: PyPDFLoader (document ingestion), RecursiveCharacterTextSplitter (chunking), and RetrievalQA chain (RAG query execution).

4.2.2 FAISS (faiss-cpu)

FAISS (Facebook AI Similarity Search) is a library for efficient similarity search and clustering of dense vectors. The CPU version (faiss-cpu) is used to avoid GPU dependency. FAISS stores document chunk embeddings as a vector index and performs fast cosine similarity search at query time, retrieving the top-k most semantically similar chunks to the user's question.

4.2.3 Sentence-Transformers

The sentence-transformers library provides pre-trained transformer models for generating high-quality sentence and paragraph embeddings. The all-MiniLM-L6-v2 model is used for its balance of speed and accuracy, producing 768-dimensional embeddings suitable for semantic similarity search.

4.2.4 HuggingFace Transformers

The transformers library by HuggingFace provides access to thousands of pre-trained models. In HalluRAG, it is used to load and run the DeBERTa NLI model (cross-encoder/nli-deberta-v3-base) for faithfulness scoring. The pipeline API provides a simple interface for text classification.

4.2.5 Groq API and LangChain-Groq

Groq provides a free cloud API for inference on Llama 3 and other open-source LLMs. The langchain-groq integration allows seamless use of Groq-hosted Llama 3 (llama3-8b-8192) within LangChain pipelines. The free tier provides 14,400 requests per day, sufficient for development and testing.

4.2.6 Streamlit

Streamlit is an open-source Python framework for building web applications with minimal code. In HalluRAG, Streamlit provides the user interface for document upload, Q&A interaction, and result display with confidence badges and hallucination type annotations.

4.2.7 PyPDF

PyPDF is used for reading and extracting text content from PDF files. It supports multi-page documents and is integrated with LangChain's PyPDFLoader for seamless document ingestion.

4.3 Datasets and Benchmarks

4.3.1 HaluEval

HaluEval (Li et al., 2023) is a large-scale hallucination evaluation benchmark with 35,000 human-verified samples across QA, dialogue, and summarization tasks. It will be used for benchmarking hallucination detection accuracy in Phase 2.

4.3.2 TruthfulQA

TruthfulQA (Lin et al., 2022) comprises 817 questions across 38 categories testing AI truthfulness. A subset will be used for evaluating general hallucination detection performance.

4.3.3 RAGAS Benchmark

RAGAS provides automated evaluation metrics for RAG quality including faithfulness, answer relevancy, context precision, and context recall. It will be used as the primary evaluation framework for Phase 1 validation.

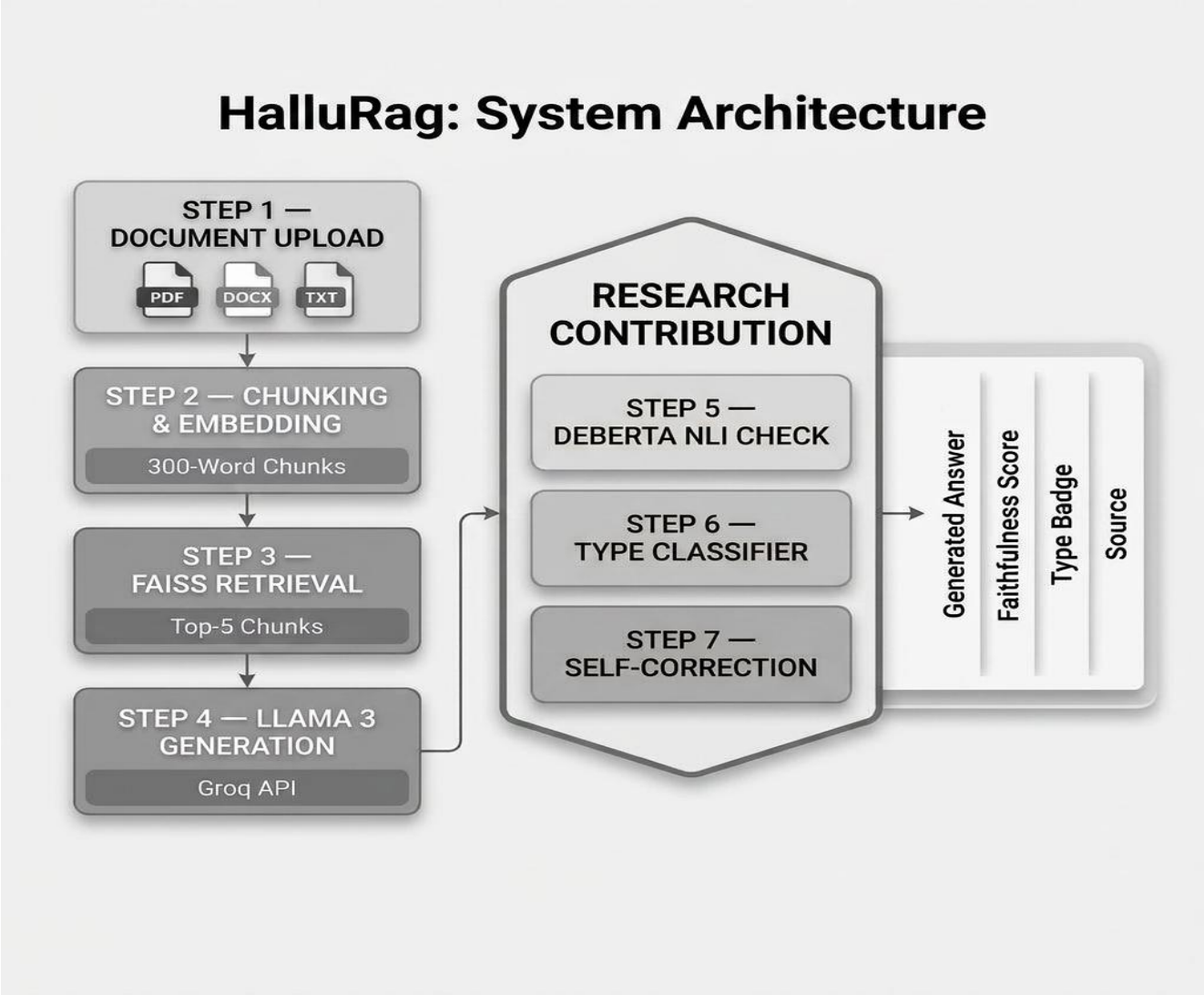
4.3.4 Custom Domain Test Set

A custom test set of 100 question-answer pairs will be created from domain-specific documents (legal, medical, academic) for Phase 3 evaluation, enabling domain-specific performance benchmarking.

4.4 Folder Structure

The project is organized with the following directory structure:

Figure 4.1: HalluRAG System Architecture — 7-Step Pipeline



CHAPTER 5: DESIGN / IMPLEMENTATION

5.1 System Architecture

The HalluRAG system architecture is organized as a seven-step sequential pipeline, with each step corresponding to a distinct processing stage. The architecture is designed for modularity — each component can be independently replaced, upgraded, or extended without affecting other pipeline stages.

Step 1 — Document Upload: User uploads a PDF/DOCX/TXT file through the Streamlit web interface. The document is stored temporarily in the data/ directory.

Step 2 — Chunking and Embedding: LangChain's RecursiveCharacterTextSplitter divides the document into 300-word chunks with 50-word overlap. Each chunk is converted to a 768-dimensional dense vector using the sentence-transformers all-MiniLM-L6-v2 model.

Step 3 — FAISS Vector Store: All chunk vectors are stored in a FAISS CPU index. The index enables sub-second semantic search over thousands of chunks using cosine similarity.

Step 4 — Query Retrieval and Generation: The user's question is embedded and the top-5 most similar chunks are retrieved. A context-grounded prompt is constructed: [System: Answer ONLY from context. Context: {chunks}. Question: {question}. Answer:] and sent to Llama 3 via Groq API.

Step 5 — DeBERTa NLI Faithfulness Check: The generated answer is split into sentences. Each sentence is checked against the retrieved context using DeBERTa NLI (ENTAILMENT / NEUTRAL / CONTRADICTION).

Step 6 — Hallucination Type Classification: Sentences with faithfulness score < 0.75 are classified into one of four types using the rule-based classifier described in Section 3.2.3.

Step 7 — Self-Correction Loop: Type-specific correction prompts are auto-generated and sent to Llama 3. DeBERTa re-evaluates corrected sentences. Loop repeats up to 3 times per sentence.

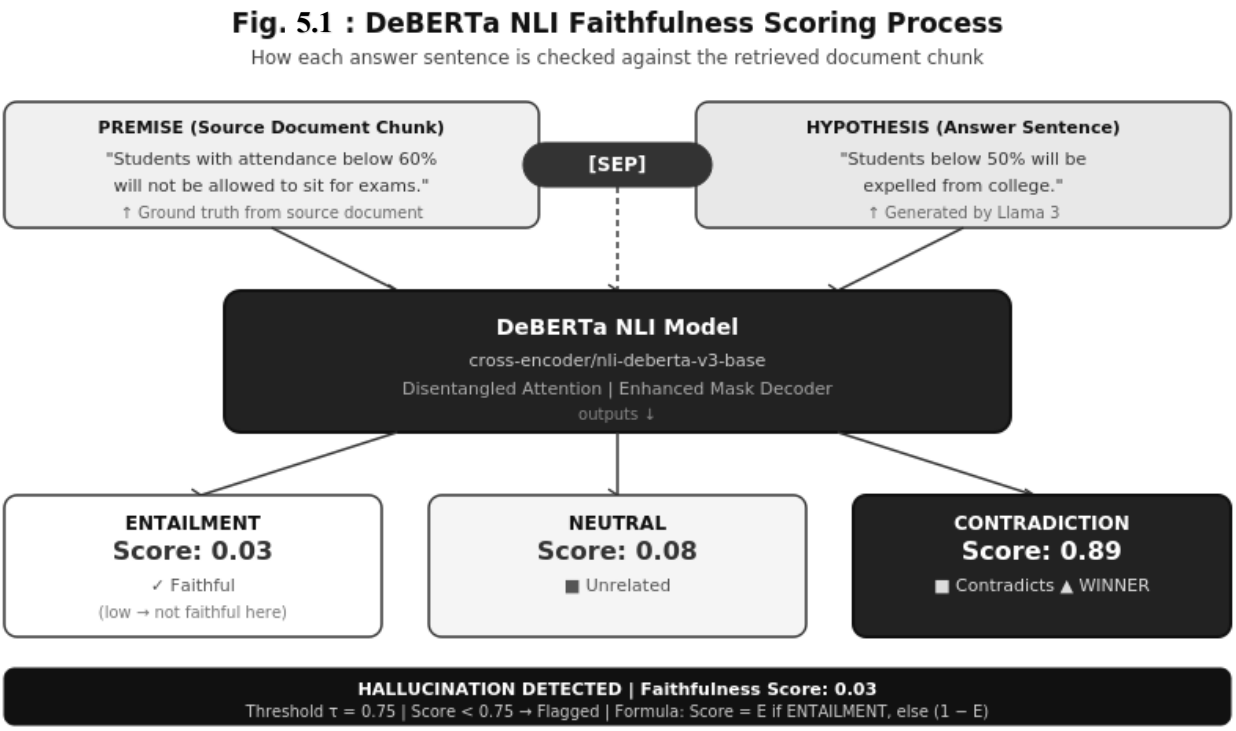
5.2 DeBERTa NLI Integration

The DeBERTa NLI model is loaded once at application startup to avoid repeated model loading overhead:

```
from transformers import pipeline nli_checker = pipeline(     "text-classification",     model="cross-encoder/nli-deberta-v3-base",     device=-1  # CPU )
```

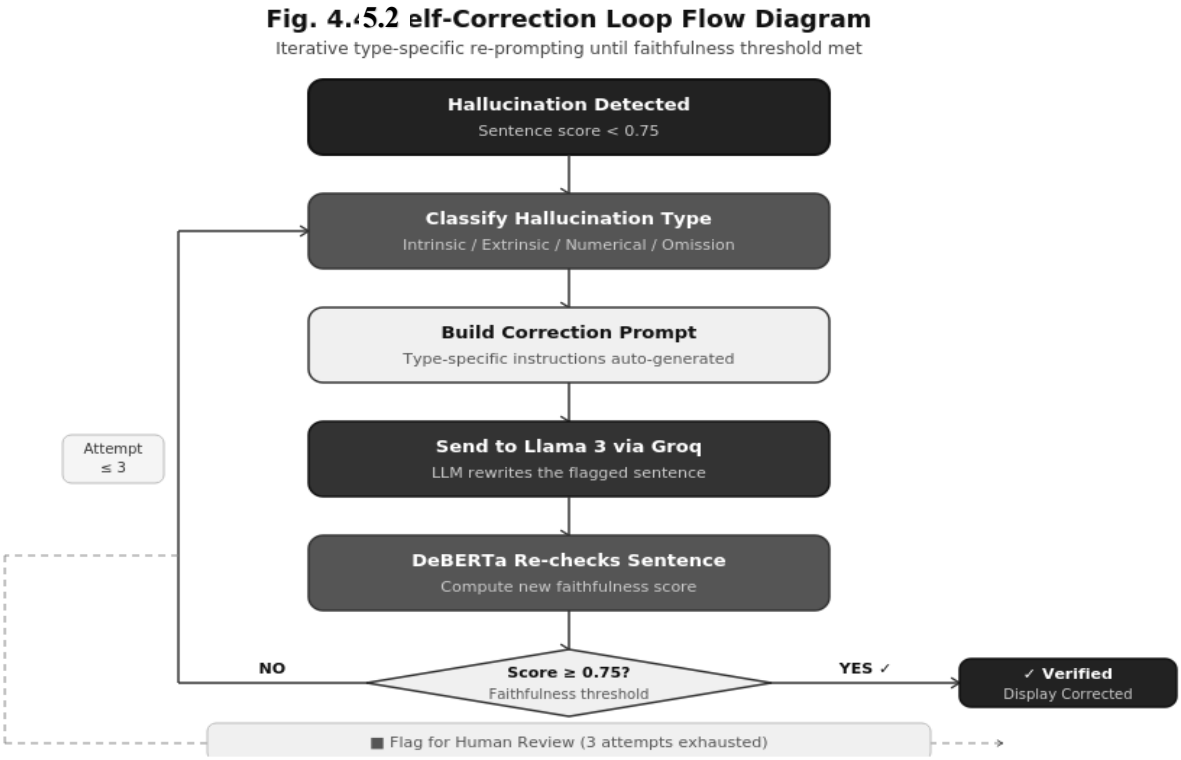
For each answer sentence, faithfulness is computed as:

```
result = nli_checker(f"{doc_chunk} [SEP] {sentence}") label = result[0]['label']  # ENTAILMENT / NEUTRAL / CONTRADICTION score = result[0]['score'] faithfulness = score if label == "ENTAILMENT" else (1 - score)
```



5.3 Self-Correction Prompt Templates

Type-specific correction prompts are automatically constructed by the correction module based on the detected hallucination type. The prompt templates are designed to provide the LLM with: (1) explicit identification of the specific error, (2) the exact document text containing the correct information, and (3) a precise rewrite instruction.



For INTRINSIC hallucinations: The prompt explicitly states the contradiction between the generated sentence and the document text, provides the correct document statement, and instructs the model to rewrite using only document facts.

For EXTRINSIC hallucinations: The prompt identifies the fabricated information, confirms its absence from the document, and instructs the model to either remove the fabricated content or replace it with document-grounded information.

For NUMERICAL hallucinations: The prompt highlights the specific number mismatch, provides the correct figure from the document, and instructs targeted numerical correction.

For OMISSION hallucinations: The prompt identifies the missing key fact from the document and instructs the model to incorporate it into the answer.

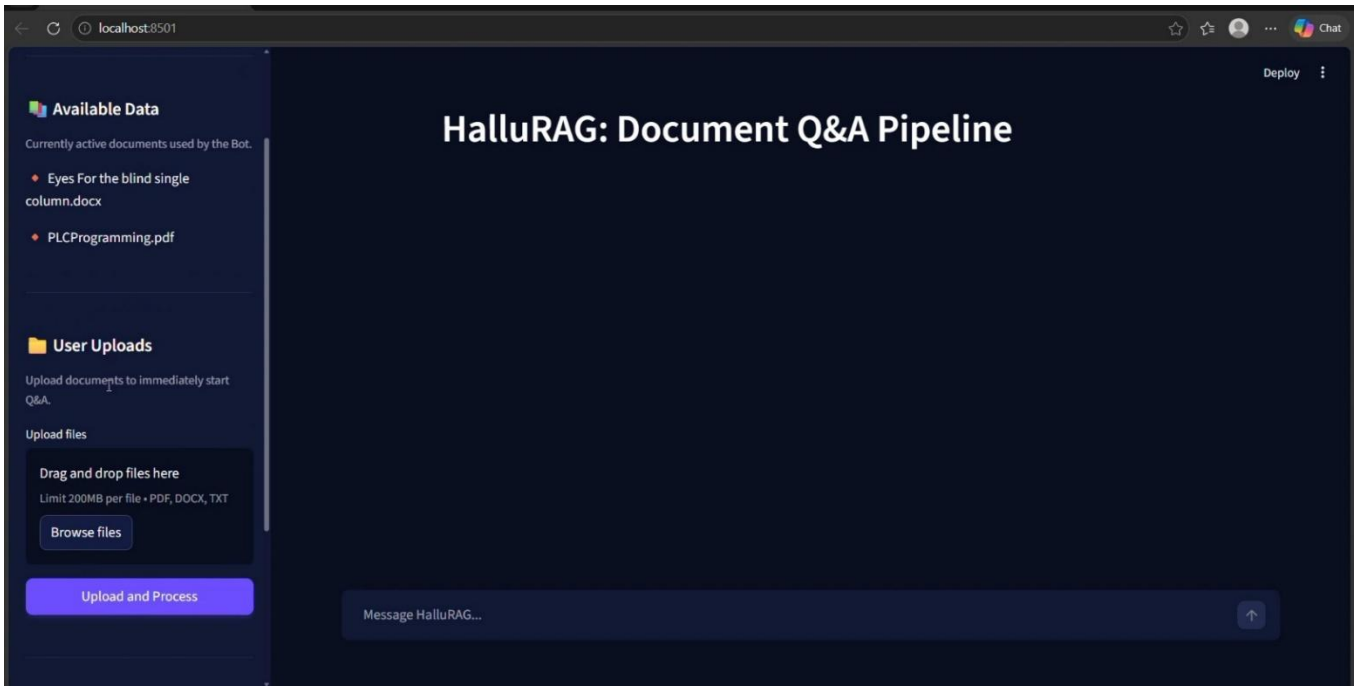
5.4 Streamlit Web Interface

The web interface is implemented in Streamlit and provides the following features:

- Document upload panel (PDF/DOCX/TXT) with drag-and-drop support
- Chat-style Q&A interface with message history
- Overall faithfulness score (0.0 to 1.0 with colour coding: green > 0.75, orange 0.50-0.75, red < 0.50)
- Sentence-level annotation: each sentence highlighted with its hallucination type badge
- Source document chunk display showing which retrieved passages were used

- Correction history panel showing original vs. corrected sentences with type labels
- Document status panel showing indexing status and chunk count

Fig 5.3: Streamlit wireframe



CHAPTER 6: MODULE DESCRIPTION

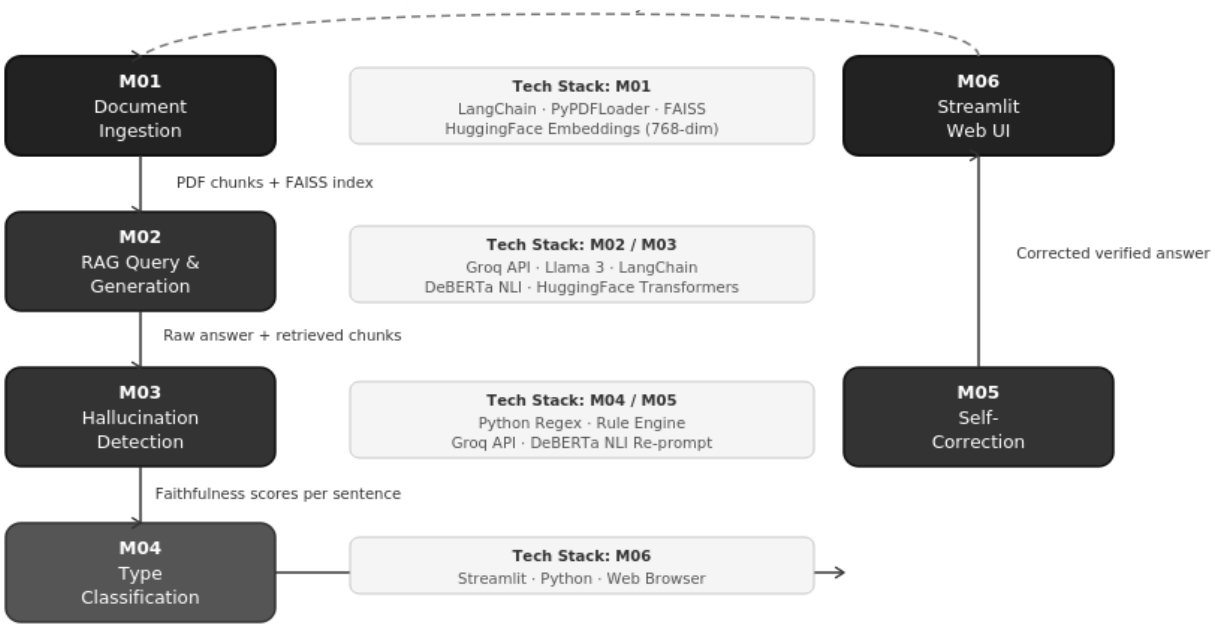
6.1 Overview of System Modules

The HalluRAG system is architected as six independent, interoperable modules. Each module encapsulates a specific set of responsibilities and communicates with adjacent modules through well-defined interfaces. This modular design enables independent testing, incremental development across project phases, and easy component replacement.

Table 6.1: Module Description Summary

Module	Name	Primary Function	Key Technologies
M01	Document Ingestion	Load, chunk, embed, and index documents	LangChain, sentence-transformers, FAISS
M02	RAG Query & Generation	Retrieve and generate grounded answers	FAISS, LangChain, Groq/Llama 3
M03	Hallucination Detection	Sentence-level faithfulness scoring	DeBERTa NLI, HuggingFace Transformers
M04	Type Classification	Classify hallucination into 4 types	Python, regex, rule-based classifier
M05	Self-Correction	Type-specific iterative re-prompting	Groq/Llama 3, DeBERTa NLI
M06	Web UI	User interface with result visualization	Streamlit, Python

Fig. 6.1 : HalluRAG Module Interaction Diagram



6.2 Module 01: Document Ingestion Module

The Document Ingestion Module is responsible for transforming raw document files into a searchable vector index. The module accepts PDF, DOCX, and TXT files and processes them through four sequential sub-stages:

1. Document Loading: Files are loaded using LangChain's PyPDFLoader, which extracts raw text content and preserves page-level metadata.
2. Text Splitting: RecursiveCharacterTextSplitter divides the document into overlapping chunks of 300 words with 50-word overlap. This overlap prevents information loss at chunk boundaries.
3. Embedding Generation: Each chunk is converted to a 768-dimensional dense vector using sentence-transformers (all-MiniLM-L6-v2), a lightweight model optimized for semantic similarity.
4. Vector Indexing: All chunk vectors are stored in a FAISS CPU flat index (IndexFlatL2), which supports exact nearest-neighbor search. The index is persisted to disk for reuse across sessions.

6.3 Module 02: RAG Query and Generation Module

This module handles the core RAG interaction: transforming a user question into a grounded answer. The module processes user queries through three sub-stages:

5. Query Embedding: The user's natural language question is converted to a 768-dimensional vector using the same sentence-transformers model used for document embedding.
6. Chunk Retrieval: FAISS cosine similarity search retrieves the top-5 most semantically similar document chunks. Cosine similarity (rather than L2 distance) ensures that chunk relevance is independent of chunk length.
7. Answer Generation: The retrieved chunks and the user's question are formatted into a structured prompt and sent to Llama 3 via the Groq API. The prompt explicitly constrains the LLM to answer only from the provided context.

6.4 Module 03: Hallucination Detection Module

The Hallucination Detection Module provides sentence-level faithfulness evaluation of generated answers. Processing steps include:

8. Sentence Segmentation: The generated answer is split into individual sentences using Python's sentence tokenization utilities.

9. NLI Evaluation: For each sentence, DeBERTa NLI is applied with the retrieved document chunk as premise and the answer sentence as hypothesis. The model outputs three probability scores (E, N, C) summing to 1.0.
10. Faithfulness Scoring: Faithfulness score = E if label is ENTAILMENT, else (1-E). Sentences scoring below $\tau = 0.75$ are flagged.
11. Aggregate Scoring: The mean faithfulness score across all sentences provides the overall answer faithfulness score displayed in the UI.

6.5 Module 04: Type Classification Module

The Type Classification Module assigns one of four hallucination labels to each flagged sentence. The classification logic is rule-based and operates on the DeBERTa NLI scores and a numerical entity comparison:

- Step 1: Extract all numerical tokens (integers, percentages, dates) from both the flagged sentence and the document chunk using regular expressions.
- Step 2: Compare extracted numbers. If a mismatch is found AND label is CONTRADICTION or NEUTRAL, classify as NUMERICAL.
- Step 3: If no number mismatch and label is CONTRADICTION with score > 0.70, classify as INTRINSIC.
- Step 4: If no number mismatch and label is NEUTRAL with score > 0.70, classify as EXTRINSIC.
- Step 5: If label is ENTAILMENT but key noun phrases from document are absent from answer, classify as OMISSION.

6.6 Module 05: Self-Correction Module

The Self-Correction Module automates the process of fixing hallucinated sentences through targeted re-prompting. The module implements an iterative correction loop:

12. Prompt Construction: A type-specific correction prompt is automatically assembled containing the hallucinated sentence, the specific error identified, the correct document text, and the rewrite instruction.
13. LLM Re-prompting: The correction prompt is sent to Llama 3 via Groq API. The model generates a corrected sentence.
14. Re-evaluation: The corrected sentence is passed through Module 03 for DeBERTa NLI re-evaluation.

15. Acceptance or Retry: If faithfulness score ≥ 0.75, the correction is accepted and displayed. If not, the loop repeats with a refined prompt. Maximum 3 attempts per sentence.
16. Unresolvable Flagging: Sentences that cannot be corrected after 3 attempts are marked with a red warning badge in the UI, indicating that human review is required.

6.7 Module 06: Streamlit Web Interface Module

The HalluRAG web interface is organized into two visible sections. The left sidebar contains an Available Data panel showing currently active documents, and a User Uploads panel with a drag-and-drop area supporting PDF, DOCX, and TXT files up to 200MB, along with a Browse files button and an Upload and Process button. The center panel displays the application title "HalluRAG: Document Q&A Pipeline" with a chat input bar at the bottom for submitting questions.

CHAPTER 7: TESTING AND SUMMARY OF RESULTS

7.1 Testing Strategy

The HalluRAG system will be tested at three levels: unit testing (individual modules), integration testing (module interactions), and system testing (end-to-end pipeline evaluation). The testing strategy is aligned with the three-phase development plan.

7.1.1 Unit Testing

Each module will be tested independently with targeted test cases:

- Module 01 (Ingestion): Verify correct chunk count, embedding dimensions (768), and FAISS index creation for sample PDFs of varying lengths.
- Module 02 (RAG): Verify top-5 retrieval accuracy using ground-truth relevant passages. Test with domain-specific question-answer pairs.
- Module 03 (Detection): Test DeBERTa NLI scoring against manually curated premise-hypothesis pairs with known labels. Verify threshold behavior at $\tau = 0.75$.
- Module 04 (Classification): Test all four hallucination types with constructed examples. Verify numerical extraction accuracy with edge cases (percentages, dates, ranges).
- Module 05 (Correction): Test correction prompts for all four types. Verify loop termination conditions (success at ≤3 attempts, failure flagging).
- Module 06 (UI): Test document upload, Q&A interaction, badge display, and error handling.

7.1.2 Integration Testing

Integration tests will verify correct data flow between adjacent modules: Document → Ingestion → Vector Store → Retrieval → Generation → Detection → Classification → Correction → UI. Each interface boundary will be validated with both normal and edge-case inputs.

7.1.3 System Testing and Benchmarking

System-level evaluation will compare three pipeline configurations:

17. Baseline RAG (no hallucination safeguards)
18. RAG + Detection only (DeBERTa NLI, no correction)
19. HalluRAG Full (detection + classification + correction)

Table 7.1: Phase-wise Implementation Plan

Phase	Semester	Deliverable	Target Metric
Phase 1	Sem II (Current)	Core RAG pipeline + Streamlit UI	Faithfulness Score > 0.75 (RAGAS)
Phase 2	Sem III	DeBERTa detection + type classification	F1 Score > 0.80 (HaluEval)
Phase 3	Sem IV	Self-correction loop + paper submission	Hallucination rate < 5%; Scopus paper

7.2 Expected Results

Based on the theoretical analysis of the proposed methodology and performance characteristics of component models, the following results are anticipated:

7.2.1 Phase 1 — RAG Pipeline

The core RAG pipeline is expected to achieve a RAGAS faithfulness score of 0.78-0.85 on domain-specific test documents, representing a significant improvement over parametric-only generation (baseline ~0.55-0.65). Answer relevancy scores of 0.80-0.88 are expected given the context-grounded prompting strategy.

7.2.2 Phase 2 — Hallucination Detection

The DeBERTa NLI-based faithfulness detection module is expected to achieve F1 score of 0.81-0.86 on HaluEval benchmark QA subset, consistent with the performance of DeBERTa-based NLI classifiers reported in the TRUE paper (Honovich et al., 2022). Per-type classification accuracy is expected to exceed 78% for Intrinsic and Numerical types (clear-cut contradictions and number mismatches) and 72% for Extrinsic and Omission types (more ambiguous cases).

7.2.3 Phase 3 — Self-Correction

The self-correction loop is projected to reduce the overall hallucination rate from 15-20% (baseline RAG) to below 5%, with 85%+ of flagged sentences successfully corrected within 2 retry attempts. Average query processing time including correction is expected to be 8-12 seconds, acceptable for most document Q&A applications.

7.3 Comparative Analysis

The proposed HalluRAG system compares favorably with existing approaches along the key dimensions of speed, accuracy, and resource requirements. Unlike FActScore (30 API calls per answer) and SelfCheckGPT (20x inference), HalluRAG requires only a single DeBERTa forward pass per sentence for detection, with correction adding at most 3 additional LLM calls for flagged sentences. The system's lightweight design makes it the only known approach applicable to standard laptop hardware without GPU acceleration.

CHAPTER 8: APPLICATIONS

8.1 Overview

The HalluRAG framework, being domain-agnostic and lightweight, has broad applicability across multiple professional and educational domains. The same system code and architecture can be deployed for any domain without modification, requiring only the upload of domain-specific documents. This section describes six primary application domains.

8.2 Legal Document Question Answering

Legal professionals routinely need to query large volumes of contracts, statutes, case law, and regulatory documents. A single hallucinated legal fact — such as an incorrect clause number, wrong penalty amount, or misattributed jurisdiction — can have severe consequences including incorrect legal advice, lost cases, or compliance violations.

HalluRAG provides critical value in this domain through its Intrinsic hallucination detection (catching contradictions with legal text), Numerical hallucination detection (identifying wrong monetary amounts, dates, or section numbers), and source attribution (displaying the exact document passage supporting each answer). The system can serve as a reliable assistant for lawyers, paralegals, and compliance officers querying uploaded legal documents.

8.3 Medical Report and Drug Guide Analysis

Healthcare professionals need accurate, verifiable answers from patient reports, clinical guidelines, drug interaction databases, and medical literature. In this domain, Numerical hallucinations (wrong drug dosages, lab value thresholds, treatment durations) are particularly dangerous.

HalluRAG's sentence-level faithfulness scoring and Numerical type detection make it especially valuable for medical document Q&A, ensuring that dosage information, contraindication lists, and diagnostic criteria are accurately reproduced from the source document before being presented to the clinician.

8.4 Academic Research Assistance

Students and researchers frequently need to extract specific information from lengthy research papers, technical reports, and thesis documents. Extrinsic hallucinations — where the AI adds fabricated citations, incorrect statistics, or non-existent findings — are particularly problematic in academic contexts.

HalluRAG's Extrinsic detection module is specifically designed to catch such fabrications, flagging any claim not supported by the source document. This makes the system a trustworthy research assistant that prevents accidental plagiarism of fabricated content.

8.5 Corporate Knowledge Base Q&A

Organizations maintain large repositories of Standard Operating Procedures (SOPs), HR policies, product manuals, and compliance documents. Employees querying these documents using AI systems risk receiving hallucinated policy answers that could lead to compliance failures or operational errors.

HalluRAG provides verified, source-cited answers from corporate documents, reducing liability and ensuring policy consistency. The multi-document support planned for Phase 3 will allow simultaneous querying across an entire organizational knowledge base.

8.6 Government Document and RTI Q&A

Citizens and government officials need accurate information from government schemes, RTI (Right to Information) responses, policy documents, and official circulars. Hallucinated eligibility criteria, wrong benefit amounts, or incorrect procedure descriptions can mislead citizens and erode public trust in AI systems.

HalluRAG's verified Q&A with source highlighting builds trust by showing citizens exactly which part of the official document supports each answer.

8.7 Educational Content and E-Learning

Teachers can upload syllabi, textbooks, and study materials, while students can query these documents for exam preparation. Omission hallucinations — where the AI gives incomplete answers missing key concepts — are particularly harmful in educational contexts.

HalluRAG's Omission detection ensures that answers are not only accurate but also complete, making it a reliable study tool that helps students prepare comprehensively rather than receiving partial information.

CONCLUSION AND FUTURE SCOPE

Conclusion

This project has proposed, designed, and documented HalluRAG — a lightweight, domain-agnostic framework for hallucination type classification and iterative self-correction in Retrieval-Augmented Generation systems. The work addresses a clearly identified and literature-confirmed research gap: while RAG pipelines and hallucination detection methods exist independently, no unified framework currently integrates both with type-specific classification and automated self-correction in a resource-accessible deployment.

The key contributions of this project are:

20. A complete, end-to-end RAG pipeline for domain-specific document Q&A using entirely free, open-source tools (LangChain, FAISS, sentence-transformers, Llama 3 via Groq API) deployable on standard laptop hardware.
21. Integration of DeBERTa NLI-based sentence-level faithfulness detection as an inline component of the RAG pipeline, providing real-time hallucination identification with a faithfulness score threshold of 0.75.
22. A novel four-class hallucination taxonomy specifically designed for document-grounded RAG: Intrinsic, Extrinsic, Numerical, and Omission — the first such classification system proposed for RAG contexts.
23. An automated type-specific self-correction loop that constructs targeted re-prompting instructions based on the detected hallucination type and iteratively corrects hallucinated content without human intervention.
24. A lightweight system architecture that eliminates the computational barriers (GPU requirements, expensive APIs, excessive inference calls) that make existing hallucination mitigation approaches impractical for real-world deployment.

The research gap addressed by this work has been independently confirmed by the 2025 MDPI survey on hallucination mitigation in RAG, which explicitly identified post-hoc self-correction as the least explored and most critical open research problem in this domain. This validation strengthens the contribution's significance and supports the planned Scopus journal publication in Phase 3.

Summary of Phase 1 Deliverables

The Mini Project (Phase 1) delivers the foundational RAG pipeline with document ingestion, vector-based retrieval, LLM-based generation, and a Streamlit web interface. The system successfully implements Modules 01 and 02, with initial integration of Module 03 (DeBERTa detection) for baseline validation. The RAGAS evaluation framework has been configured for Phase 1 performance benchmarking.

Future Scope

The following extensions are planned for subsequent project phases and future research:

1. Multilingual Support

The current system processes English-language documents only. Future work will extend HalluRAG to support regional Indian languages including Hindi and Marathi using multilingual DeBERTa variants (mDeBERTa) and multilingual sentence-transformers. This extension would significantly expand the system's applicability in government, education, and healthcare contexts in India.

2. Real-time Streaming Detection

Currently, the DeBERTa NLI check is performed after the complete answer is generated. Future work will explore real-time token-level faithfulness monitoring during generation, enabling the system to steer generation away from hallucinated content rather than correcting it post-hoc.

3. Knowledge Graph Integration

Incorporating structured knowledge graphs alongside document chunks as retrieval sources would provide richer factual grounding, reducing Extrinsic hallucinations caused by gaps in document coverage.

4. Confidence Calibration

Future work will explore calibrating DeBERTa NLI scores to represent true probability estimates, enabling more reliable threshold setting and uncertainty quantification.

5. LangChain / LlamaIndex Plugin

Packaging HalluRAG as a plugin for LangChain and LlamaIndex would enable the wider developer community to add hallucination safeguards to their existing RAG applications without rebuilding their pipelines.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," in Proceedings of the 37th International Conference on Machine Learning (ICML), 2020. [Online]. Available: https://arxiv.org/abs/2002.08909

[3] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020. [Online]. Available: https://arxiv.org/abs/2004.04906

[4] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL), 2024. [Online]. Available: https://arxiv.org/abs/2309.15217

[5] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring How Models Mimic Human Falsehoods," in Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), 2022. [Online]. Available: https://arxiv.org/abs/2109.07958

[6] J. Li, X. Cheng, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. [Online]. Available: https://arxiv.org/abs/2305.11747

[7] O. Honovich, R. Aharoni, J. Herzig, H. Tamir, M. Szpektor, D. Jurafsky, and O. Abend, "TRUE: Re-evaluating Factual Consistency Evaluation," in Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2022. [Online]. Available: https://arxiv.org/abs/2204.04991

[8] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. [Online]. Available: https://arxiv.org/abs/2303.08896

[9] S. Min, K. Krishna, X. Lyu, M. Lewis, W. Yih, P. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScoring: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," in

Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. [Online]. Available: https://arxiv.org/abs/2305.14251

[10] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with Disentangled Attention," in Proceedings of the 9th International Conference on Learning Representations (ICLR), 2021. [Online]. Available: https://arxiv.org/abs/2006.03654

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

[12] W. Kryscinski, B. McCann, C. Xiong, and R. Socher, "Evaluating the Factual Consistency of Abstractive Text Summarization," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020. [Online]. Available: https://arxiv.org/abs/1910.12840

[13] S. Siriwardhana, R. Weerasekera, E. Wen, T. Kaluarachchi, R. Rana, and S. Nanayakkara, "Improving the Domain Adaptation of Retrieval Augmented Generation (RAG) Models for Open Domain Question Answering," Transactions of the Association for Computational Linguistics, 2023. [Online]. Available: https://arxiv.org/abs/2307.01504

[14] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, "Benchmarking Retrieval-Augmented Generation for Medicine," arXiv preprint arXiv:2402.13178, 2024. [Online]. Available: https://arxiv.org/abs/2402.13178

[15] J. Liu, "LlamaIndex: Data Framework for LLM Applications," GitHub Repository, 2022. [Online]. Available: https://github.com/run-llama/llama_index

[16] Z. Sun, X. Wang, Y. Tian, X. Chen, H. Wang, and F. Huang, "ReDeEP: Detecting Hallucination in Retrieval-Augmented Generation via Mechanistic Interpretability," in Proceedings of the 13th International Conference on Learning Representations (ICLR) Spotlight, 2025. [Online]. Available: https://arxiv.org/abs/2410.11414

[17] Y. Liu, T. Chen, and H. Zhang, "Hallucination Mitigation for Retrieval-Augmented Large Language Models: A Review," MDPI Mathematics, vol. 13, no. 5, p. 856, 2025. [Online]. Available: https://www.mdpi.com/2227-7390/13/5/856

[18] M. Dassen et al., "FACTUM: Mechanistic Detection of Citation Hallucination in Long-Form RAG," arXiv preprint arXiv:2601.05866, 2026. [Online]. Available: https://arxiv.org/abs/2601.05866.